4. System Design

The system follows a modular architecture where the frontend communicates with a Python-based Flask backend. The backend serves the trained Machine Learning model and queries local CSV-based knowledge bases to retrieve auxiliary medical information.

4.1 Use Case Diagram

The following diagram illustrates the interactions between the primary actors and the system's core functionalities.

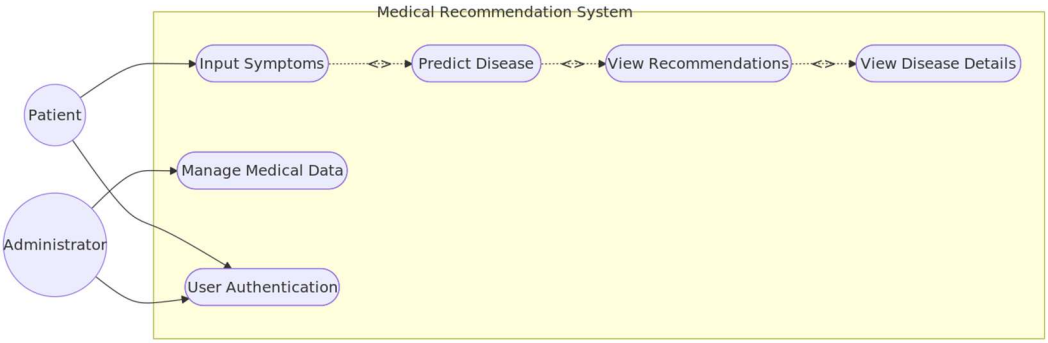

Figure 1: Use Case Diagram for the Medical Recommendation System

4.2 Architecture Components

- **Data Layer:** Contains CSV files (Symptom_severity.csv, medications.csv, precautions_df.csv, etc.) which act as the system's knowledge base [1].
- **Processing Layer:** Utilizes Scikit-learn for model inference and Pandas for data manipulation.
- **Presentation Layer:** A Flask-based web application that renders HTML templates for the user.

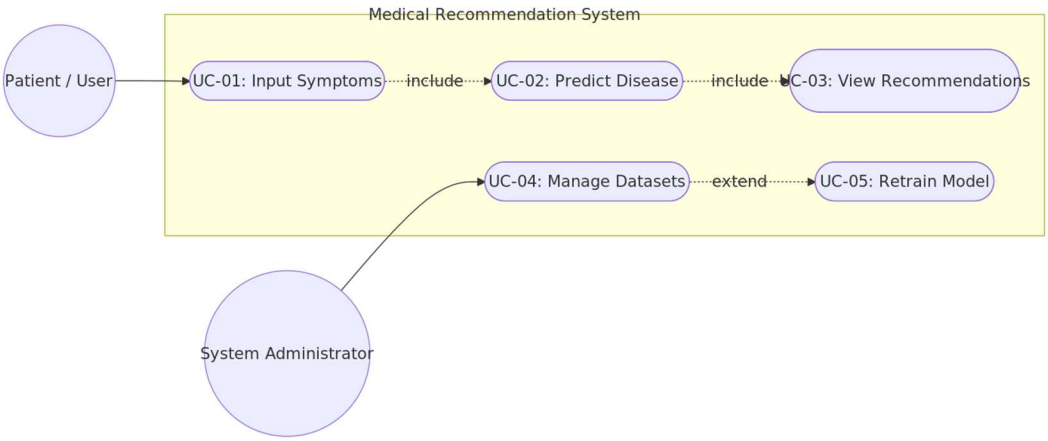

Figure 4.1: Use Case Diagram — Medical Recommendation System

The table below provides formal descriptions for each use case.

Use Case ID	Name	Actor(s)	Description	Relationship

UC-01	Input Symptoms	Patient / User	The user selects or types one or more symptoms from the available symptom vocabulary.	—
UC-02	Predict Disease	System	The system vectorizes the symptom input and invokes the SVC model to produce a disease prediction.	«include» UC-01
UC-03	View Recommendations	Patient / User	The predicted disease is used to retrieve and display medications, diet, precautions, and workout information.	«include» UC-02
UC-04	Manage Datasets	System Administrator	The administrator uploads or modifies the CSV knowledge-base files (medications, diet, precautions, etc.).	—
UC-05	Retrain Model	System Administrator	Following a dataset update the administrator triggers model retraining and serialises the new pickle file.	«extend» UC-04

4.3 Functional & Non-Functional Requirements

Requirements are catalogued in accordance with IEEE 830 conventions. Functional requirements define *what* the system shall do; non-functional requirements constrain *how well* it shall do it.

4.3.1 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall provide a multi-select symptom input widget populated from the training vocabulary.	High
FR-02	The system shall convert the selected symptoms into a binary feature vector of length equal to the total symptom count.	High
FR-03	The system shall invoke the serialised SVC model to classify the symptom vector and return the predicted disease label.	High
FR-04	The system shall retrieve the disease description from description.csv and display it on the results page.	High
FR-05	The system shall retrieve recommended medications from medications.csv and present them in a modal dialogue.	High
FR-06	The system shall retrieve dietary recommendations from diets.csv and present them in a modal dialogue.	High
FR-07	The system shall retrieve precautionary measures from precautions_df.csv and present them in a modal dialogue.	High
FR-08	The system shall retrieve workout recommendations from	Medium

	workout_df.csv and present them on the results page.	
FR-09	The Flask controller shall expose a POST /predict route that accepts symptom data and returns a JSON recommendation payload.	High
FR-10	The UI shall render all recommendation categories via Bootstrap modal components to preserve page clarity.	Medium

4.3.2 Non-Functional Requirements

ID	Category	Requirement	Metric
NFR-01	Performance	The system shall return a prediction and full recommendation set within two seconds of form submission.	Response time < 2 s (95th percentile)
NFR-02	Accuracy	The SVC model shall achieve a classification accuracy of no less than 90% on the held-out test set.	Accuracy ≥ 90%
NFR-03	Usability	The interface shall be fully responsive across desktop, tablet, and mobile viewports using Bootstrap 4.	Passes Bootstrap breakpoint tests at 576 px, 768 px, 992 px, 1200 px
NFR-04	Reliability	The system shall handle invalid or empty symptom submissions gracefully with user-facing error	Zero unhandled exceptions in production logs

		messages and HTTP 400 responses.	
NFR-05	Maintainability	Medical knowledge-base updates shall require only CSV file replacement with no code changes.	Dataset refresh time < 5 minutes
NFR-06	Security	All user inputs shall be sanitised server-side to prevent injection attacks before processing.	OWASP Top-10 compliant input handling

4.4 Software Architecture — MVC Pattern

The system adheres to the Model-View-Controller (MVC) architectural pattern. The **Model** layer encapsulates the SVC machine-learning model and the CSV knowledge bases. The **View** layer consists of HTML/Bootstrap Jinja2 templates rendered by Flask. The **Controller** layer is implemented in app.py, which mediates all interactions between the View and the Model.

```
flowchart TD subgraph VIEW ["View Layer (HTML / Bootstrap Templates)"] V1[index.html — Symptom Input Form] V2[results.html — Prediction & Recommendations] V3[Bootstrap Modal Components] end subgraph CONTROLLER ["Controller Layer (Flask app.py)"] C1[Route: GET /] C2[Route: POST /predict] C3[Preprocessor — Symptom Vectorizer] end subgraph MODEL ["Model Layer (ML Model + CSV Databases)"] M1[SVC Model — svc.pkl] M2[description.csv] M3[medications.csv] M4[diets.csv] M5[precautions_df.csv] M6[workout_df.csv] end V1 -->|HTTP POST /predict| C2 C1 -->|render_template| V1 C2 --> C3 C3 -->|feature vector| M1 M1 -->|predicted disease label| C2 C2 -->|query by disease| M2 C2 -->|query by disease| M3 C2 -->|query by disease| M4 C2 -->|query by disease| M5 C2 -->|query by disease| M6 C2 -->|render_template + data| V2 V2 --> V3
```

Figure 4.2: MVC Architecture of the Medical Recommendation System

4.5 Database Design & Data Modelling

4.5.1 Entity-Relationship Diagram

Although the system employs flat-file CSV storage, the logical data model is expressed below as an Entity-Relationship (ER) diagram, simulated using a Mermaid flowchart. Entities are shown as rectangles, primary keys are underlined in the schema tables, and relationships are labelled on connecting arrows.

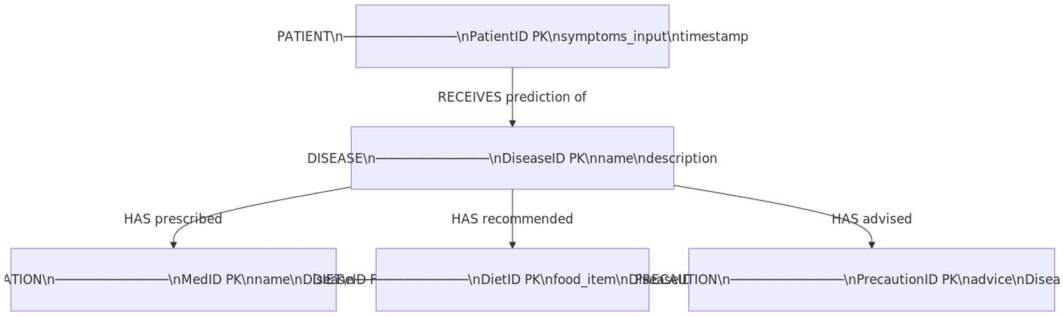

Figure 4.3: Entity-Relationship Diagram (ER) — Logical Data Model

4.5.2 Logical & Physical Schema

The following tables define the physical schema for each data entity.

Table: PATIENT

Column	Data Type	Constraint	Description
PatientID	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique session identifier
symptoms_input	VARCHAR(500)	NOT NULL	Comma-separated symptom string submitted by user
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of query submission

Table: DISEASE

Column	Data Type	Constraint	Description
DiseaseID	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique disease identifier
name	VARCHAR(200)	NOT NULL, UNIQUE	Canonical disease name (matches model label)
description	TEXT	NOT NULL	Clinical description of the disease

Table: MEDICATION

Column	Data Type	Constraint	Description
MedID	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique medication record identifier
name	VARCHAR(200)	NOT NULL	Medication name
DiseaseID	INTEGER	FOREIGN KEY → DISEASE(DiseaseID)	Associated disease

Table: DIET

Column	Data Type	Constraint	Description
DietID	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique diet record identifier
food_item	VARCHAR(200)	NOT NULL	Recommended food or dietary item
DiseaseID	INTEGER	FOREIGN KEY → DISEASE(DiseaseID)	Associated disease

Table: PRECAUTION

Column	Data Type	Constraint	Description
PrecautionID	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique precaution record identifier
advice	VARCHAR(500)	NOT NULL	Clinical precautionary advice text
DiseaseID	INTEGER	FOREIGN KEY → DISEASE(DiseaseID)	Associated disease

4.6 Data Flow Diagram (DFD)

4.6.1 Level 0 — Context Diagram

The context diagram presents the system as a single black-box process interacting with two external entities: the *User* (who supplies symptom input) and the *CSV Knowledge Bases* (which provide medical data). The sole output to the user is the consolidated recommendation set.

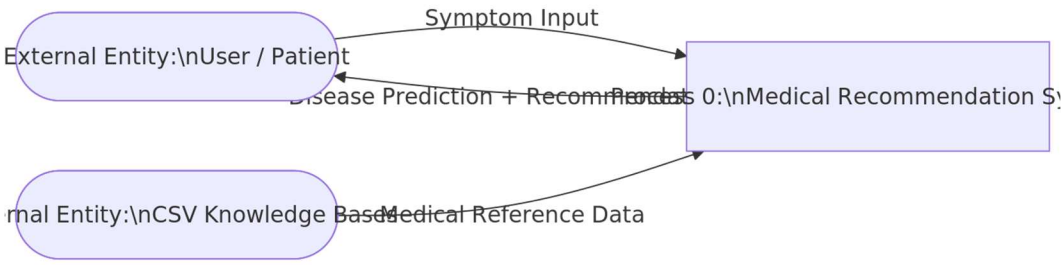

Figure 4.4: DFD Level 0 — Context Diagram

4.6.2 Level 1 — Detailed Data Flow

The Level 1 DFD decomposes the central process into its constituent processing stages, revealing the internal data flows between the symptom parser, the vectorizer, the ML inference engine, the database lookup module, and the result formatter.

Figure 4.5: DFD Level 1 — Detailed Data Flow

4.7 Sequence Diagram

The sequence diagram below, simulated as a top-down flowchart, depicts the chronological interaction between the system components for a single predict request cycle. Each labelled arrow represents a message or method invocation between participating components.

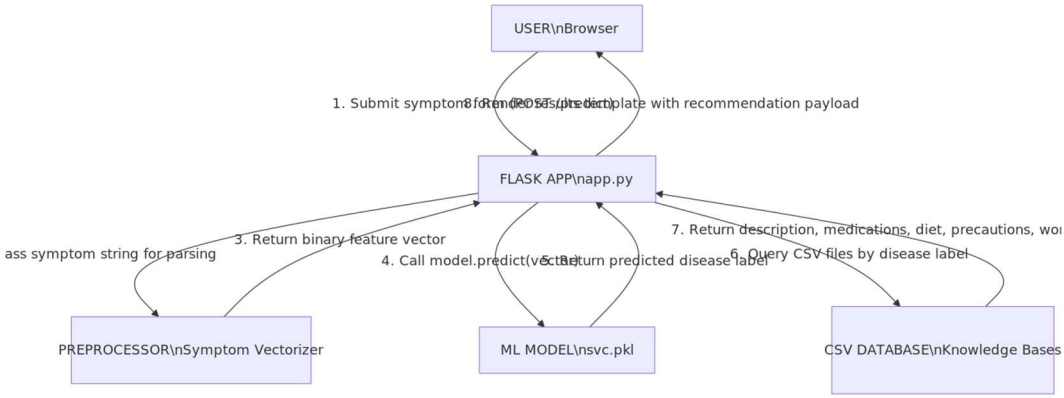

Figure 4.6: Sequence Diagram — Predict Request Lifecycle (simulated as flowchart)

4.8 Activity Diagram

The activity diagram below models the end-to-end behavioural workflow of the system from session initiation to result display. A decision node evaluates whether the submitted symptoms constitute a valid, non-empty input before the system proceeds to the ML pipeline.

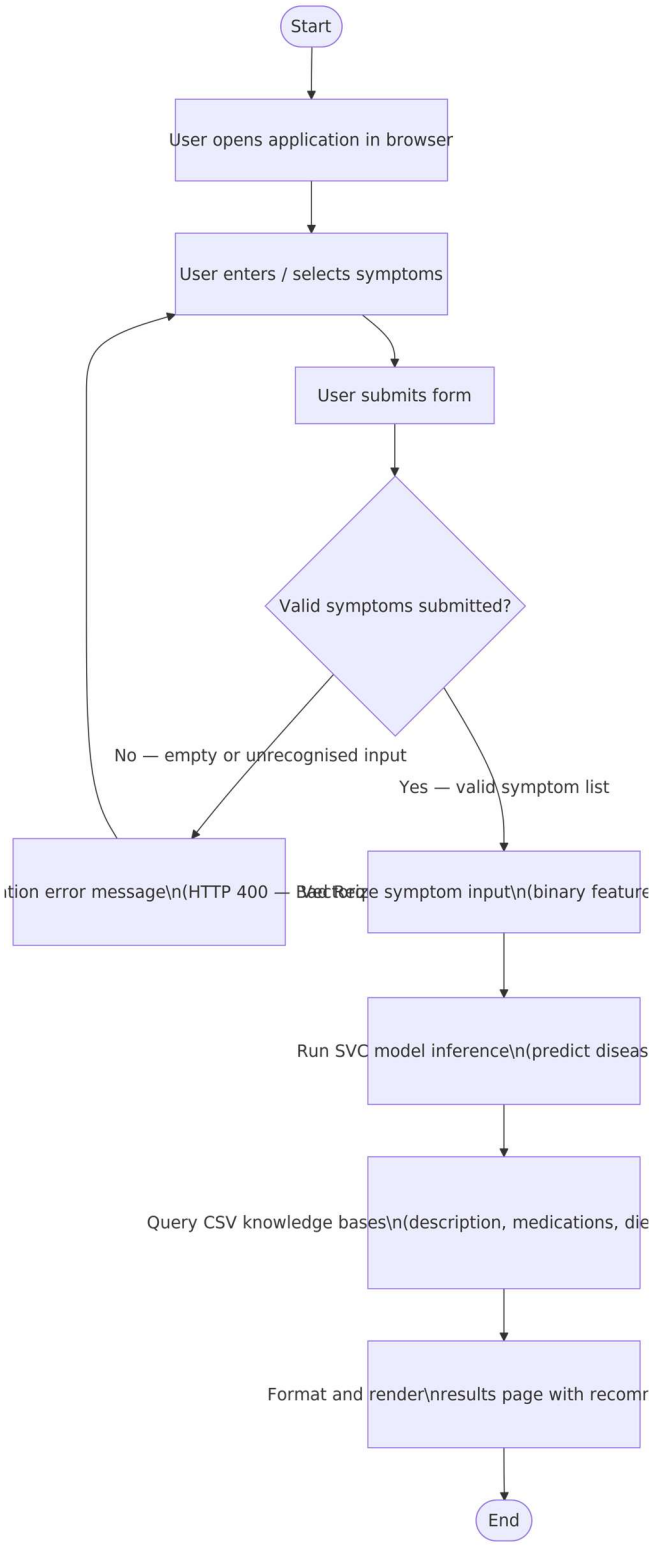

Figure 4.7: Activity Diagram — System Workflow

4.9 State Diagram

The state diagram characterises the discrete states of the system during a user interaction session. Each state transition is labelled with the triggering event or condition.

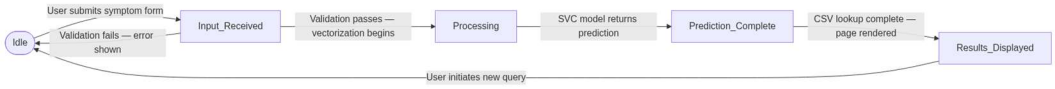

Figure 4.8: State Diagram — System Session States

4.10 Class Diagram

The class diagram below, simulated using a Mermaid flowchart, depicts the principal software classes of the system, their attributes and methods, and the dependency relationships between them.

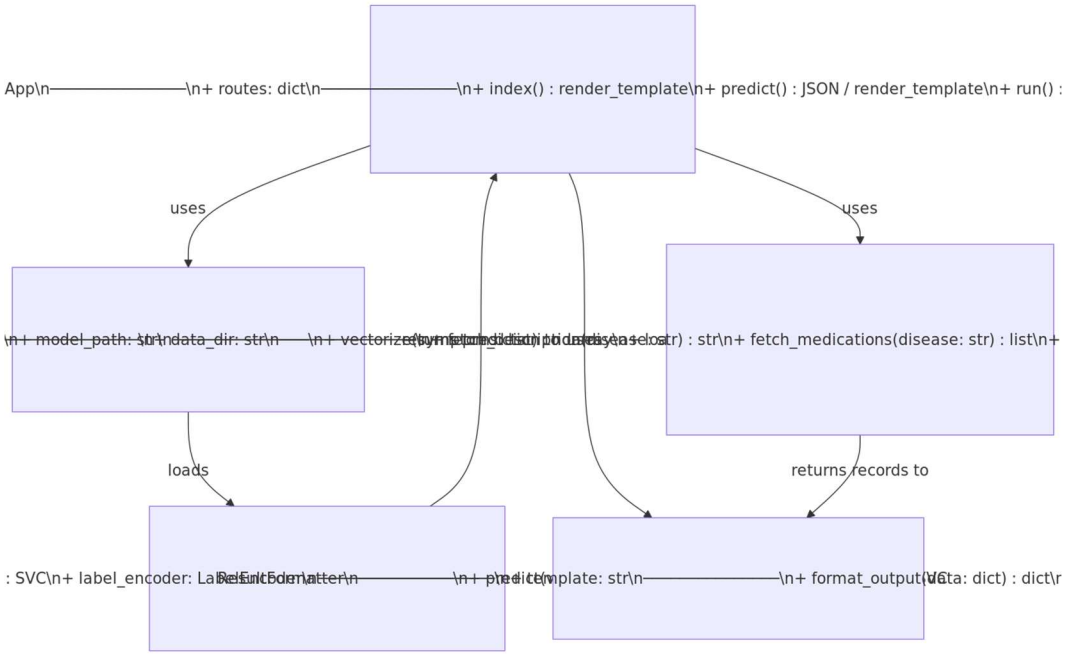

Figure 4.9: Class Diagram — Principal System Classes (simulated as flowchart)

4.11 UI/UX Design & Wireframes

The following wireframe table describes the layout of the primary application screen. The interface is constructed using the Bootstrap 4 grid system and adheres to WCAG 2.1 Level AA accessibility guidelines.

Region	Component	Description
Header (full width)	Navigation Bar	System title "MedRecommend" in medical blue (#2980b9); Bootstrap navbar-dark; no authentication links in MVP.

Main — Left column (col-md-6)	Symptom Input Area	Multi-select dropdown or tag-input populated from the training vocabulary. Label: "Select Your Symptoms". Minimum 1, maximum 17 symptoms selectable.
Main — Left column (col-md-6)	Predict Button	Full-width Bootstrap btn-success button labelled "Get Prediction". Triggers POST /predict on click.
Main — Right column (col-md-6)	Results Section	Displays predicted disease name in an alert box (alert-info) and disease description paragraph.
Main — Right column (col-md-6)	Recommendation Modal Buttons	Four Bootstrap outline buttons: "Medications", "Diet", "Precautions", "Workout" — each opens a Bootstrap modal with the corresponding recommendation list.
Footer (full width)	Disclaimer	Static text: "This system is for informational purposes only and does not constitute professional medical advice." Font size 10pt, centred.

UI/UX Guidelines:

- **Colour Scheme:** Primary background white (#FFFFFF); primary accent medical blue (#2980b9); call-to-action green (#27ae60); error red (#e74c3c).
- **Typography:** Body text — Arial / Roboto, 12pt; Headings — Roboto Bold; Line-height 1.6 for readability.
- **Accessibility:** WCAG 2.1 Level AA compliance; minimum contrast ratio 4.5:1; all interactive elements keyboard-navigable; ARIA labels on modal triggers.
- **Responsiveness:** Bootstrap 4 grid system; fluid container; breakpoints at 576 px (sm), 768 px (md), 992 px (lg), and 1200 px (xl).
- **Feedback:** Spinner overlay displayed during POST /predict to communicate processing state.

4.12 Technology Stack

The following table catalogues every technology employed in the system, organised by architectural layer, with a brief rationale for its selection.

Layer	Technology	Version	Purpose
Frontend	HTML5	5	Semantic document structure and form elements for symptom input.
Frontend	CSS3 / Bootstrap	4.x	Responsive grid layout, modal components, and consistent visual styling.
Backend	Python	3.9+	Primary implementation language for server-side logic and ML pipeline.
Backend	Flask	2.x	Lightweight WSGI micro-framework for HTTP routing and template rendering.
ML Engine	Scikit-learn (SVC)	1.x	Support Vector Classifier for disease prediction from symptom vectors.
ML Engine	NumPy	1.24+	Numerical array operations; feature vector construction.
ML Engine	Pandas	1.5+	CSV ingestion, dataframe querying, and data manipulation.
Model Storage	Pickle (.pkl)	Standard Library	Serialisation and deserialisation of the trained SVC model object.
Data Storage	CSV Flat Files	—	Lightweight structured storage for medical

			knowledge bases (descriptions, medications, diets, precautions, workouts).
Development IDE	Jupyter Notebook / VS Code	Latest	Interactive model training (Jupyter) and application development (VS Code).
Version Control	Git / GitHub	Latest	Source code management and collaborative development.

4.13 Deployment Diagram

The deployment diagram illustrates the physical distribution of system artefacts across runtime nodes. In the standard deployment configuration the user's browser communicates over HTTP(S) with a Flask development or production server, which in turn loads the serialised model from disk and queries the CSV knowledge bases stored on the same server node.

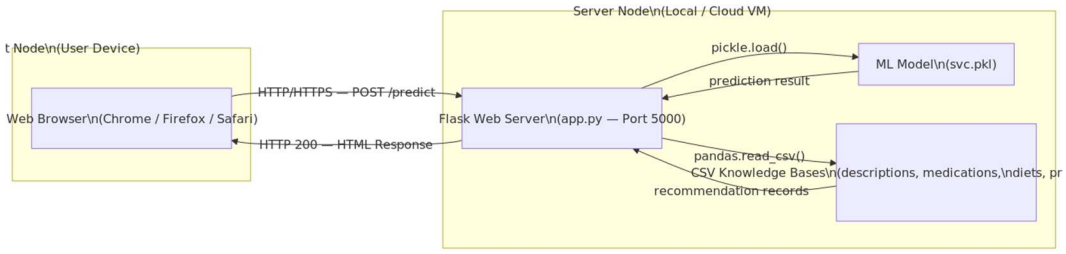

Figure 4.10: Deployment Diagram — System Runtime Architecture

4.14 Component Diagram

The component diagram decomposes the system into its logical constituent components, grouped by architectural layer. Directed arrows represent provided/required interface dependencies between components.

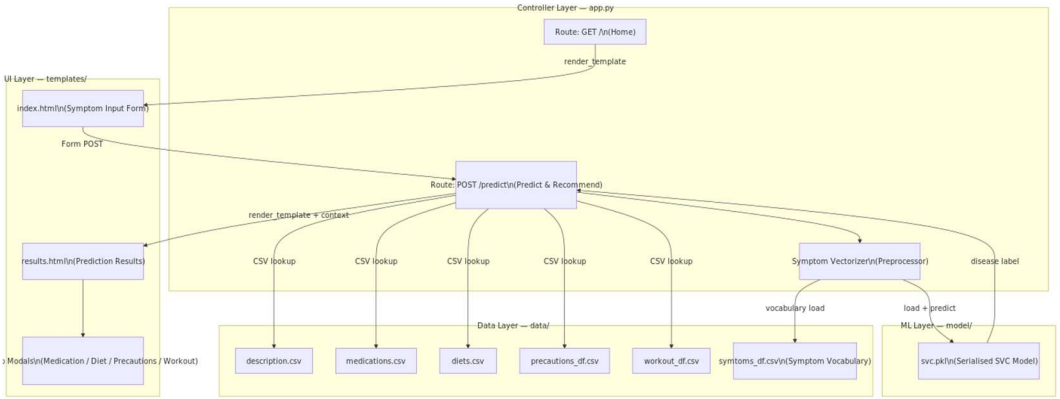

Figure 4.11: Component Diagram — Layered System Components

4.15 API Documentation

The system exposes a single RESTful HTTP endpoint. The table below documents the endpoint contract, including the request schema, response schema, and example payloads.

Attribute	Detail
Endpoint URL	POST /predict
Content-Type (Request)	application/x-www-form-urlencoded or application/json
Content-Type (Response)	application/json or text/html (template render)
Authentication	None (MVP phase — open endpoint)
Rate Limiting	Not enforced in local deployment; recommend 60 req/min in production

Request Body (JSON):

Field	Type	Required	Description	Example Value
symptoms	string	Yes	Comma-separated list of symptom names matching training vocabulary	"itching, skin_rash, fever"

Response Body (JSON — HTTP 200 OK):

Field	Type	Description	Example Value
disease	string	Predicted disease label	"Fungal infection"
description	string	Clinical description of the predicted disease	"A fungal infection, also called mycosis..."

medications	array[string]	List of recommended medications	["Antifungal Cream", "Fluconazole"]
diet	array[string]	List of dietary recommendations	["Garlic", "Yogurt", "Coconut oil"]
precautions	array[string]	List of precautionary measures	["Keep the affected area dry", "Avoid sharing towels"]
workout	array[string]	Suggested workout or physical activity guidance	["Light stretching", "Avoid public pools"]

Error Response (HTTP 400 Bad Request):

Field	Type	Description
error	string	Human-readable error message, e.g. "No valid symptoms provided. Please enter at least one recognised symptom."

4.16 Testing & Validation Plan

The testing strategy follows a layered approach comprising unit tests, integration tests, and user acceptance testing (UAT). All test cases are mapped to the functional requirements defined in Section 4.3.

Test Type	Test Case ID	Test Case Description	Expected Result	Maps to FR	Status
Unit	UT-01	Symptom vectorizer — valid input: ["itching", "fever"]	Returns binary ndarray of correct length with 1s at correct indices	FR-02	Planned
Unit	UT-02	Symptom vectorizer — empty input: []	Raises ValueError or	FR-02	Planned

			returns zero-vector with warning		
Unit	UT-03	SVC model prediction — known symptom vector	Returns correct disease string label as per training ground truth	FR-03	Planned
Unit	UT-04	DatabaseManager.fetch_medications("Fungal infection")	Returns non-empty list of medication strings	FR-05	Planned
Unit	UT-05	DatabaseManager.fetch_diet("Diabetes")	Returns non-empty list of diet strings	FR-06	Planned
Unit	UT-06	ResultFormatter.format_output — valid data dict	Returns well-formed JSON-serialisable dict with all required keys	FR-09	Planned
Integration	IT-01	POST /predict with valid JSON body {symptoms: "itching, fever"}	HTTP 200; response JSON contains all six fields: disease, description, medications, diet, precautions, workout	FR-03 – FR-09	Planned
Integration	IT-02	POST /predict with empty body	HTTP 400; response JSON contains "error" field	FR-09, NFR-04	Planned

Integration	IT-03	GET / — home page loads	HTTP 200; response HTML contains symptom input widget	FR-01	Planned
Integration	IT-04	Full round-trip: symptom input → prediction → modal display	All four Bootstrap modals populate with data from CSV lookup	FR-04 – FR-10	Planned
UAT	UAT-01	Non-technical user enters 3 symptoms and reads results within 2 minutes	User successfully identifies predicted disease and opens at least one recommendation modal without assistance	NFR-03	Planned
UAT	UAT-02	Mobile device (375 px viewport) — complete predict workflow	All UI elements visible and usable without horizontal scrolling	NFR-03	Planned
Performance	PT-01	Load test: 50 concurrent POST /predict requests	95th-percentile response time ≤ 2 s; no HTTP 5xx errors	NFR-01	Planned

4.17 Deployment Strategy

The deployment lifecycle progresses through three sequential environments: *Local Development*, *Staging / QA*, and *Production*. The following numbered procedure describes the complete deployment pipeline from a clean environment.

1. Environment Preparation (Local). Clone the project repository from GitHub. Create and activate a Python virtual environment: python -m venv venv && source venv/bin/activate (Linux/macOS) or venv\Scripts\activate (Windows).

2. Dependency Installation. Install all required Python packages by executing pip install -r requirements.txt. Core dependencies include Flask, Scikit-learn, NumPy, and Pandas.

3. Model & Data Verification. Confirm that svc.pkl and all CSV knowledge-base files are present in their designated directories (model/ and data/ respectively). If the model file is absent, execute the training notebook to regenerate and serialise it.

4. Local Development Server. Launch the Flask application with flask run or python app.py. The development server is accessible at http://127.0.0.1:5000. Verify correct operation using the manual test cases in Section 4.16 (IT-01 through IT-04).

5. Staging Environment. Deploy to a cloud-hosted staging instance (e.g. Heroku Review App or an AWS EC2 t2.micro instance). Configure environment variables (FLASK_ENV=staging, SECRET_KEY). Execute the full integration and UAT test suite against the staging URL.

6. Production Deployment — Heroku. Create a Procfile containing web: gunicorn app:app. Push the repository to Heroku using git push heroku main. Set production environment variables via the Heroku CLI: heroku config:set FLASK_ENV=production.

7. Production Deployment — AWS Elastic Beanstalk. Alternatively, initialise an Elastic Beanstalk application with eb init and deploy with eb create && eb deploy. Configure the application server (Gunicorn) via .ebextensions/python.config.

8. Post-Deployment Monitoring. Enable application logging (Flask logging module or cloud provider native logging). Monitor endpoint response times and error rates. Schedule periodic model retraining cycles (monthly or upon significant dataset updates) as described in UC-05.

5. Implementation Details

The implementation phase focuses on transforming raw medical data into a predictive model. The development environment utilizes Python 3.x with key libraries including Scikit-learn, Pandas, and NumPy.

5.1 Data Pre-processing

Medical datasets often contain categorical data. During implementation, the symptoms are converted into numerical formats using techniques such as **Label Encoding** or **One-Hot Encoding** to make them compatible with ML algorithms [1]. The dataset is typically split into a training set (80%) and a testing set (20%).

5.2 Model Selection and Training

Multiple classifiers are evaluated to determine the best performance. The **Support Vector Classifier (SVC)** is frequently selected for this project due to its effectiveness in high-dimensional spaces [1].

Step	Process	Description
1	Dataset Loading	Loading symptom and disease datasets using Pandas.

2	Feature Engineering	Mapping symptoms to columns and creating a binary vector for user inputs.
3	Model Training	Fitting the SVC model on the encoded training data.
4	Recommendation Logic	Using the predicted class to index into medication and diet dataframes.

5.3 Deployment

The final model is serialized using the `pickle` library, allowing the Flask application to load the pre-trained weights without retraining on every request. This ensures the system remains responsive and efficient during real-time user interaction [1].

6. Conclusion

The Personalized Medical Recommendation System represents a significant step toward accessible digital healthcare. By integrating Machine Learning with a multi-faceted recommendation engine, the system provides users with immediate insights that extend beyond simple diagnosis to include lifestyle and dietary guidance. Future enhancements could include the integration of Real-time API data for pharmacy locations and the adoption of Deep Learning models for even higher diagnostic precision.

References

1. *Medicine Recommendation System | Personalized Medical Recommendation System with Machine Learning.* [Video File]. Available at: https://www.youtube.com/watch?v=1xHU20MgvqI. (Referenced for System Architecture, Data Processing, and Technology Stack).